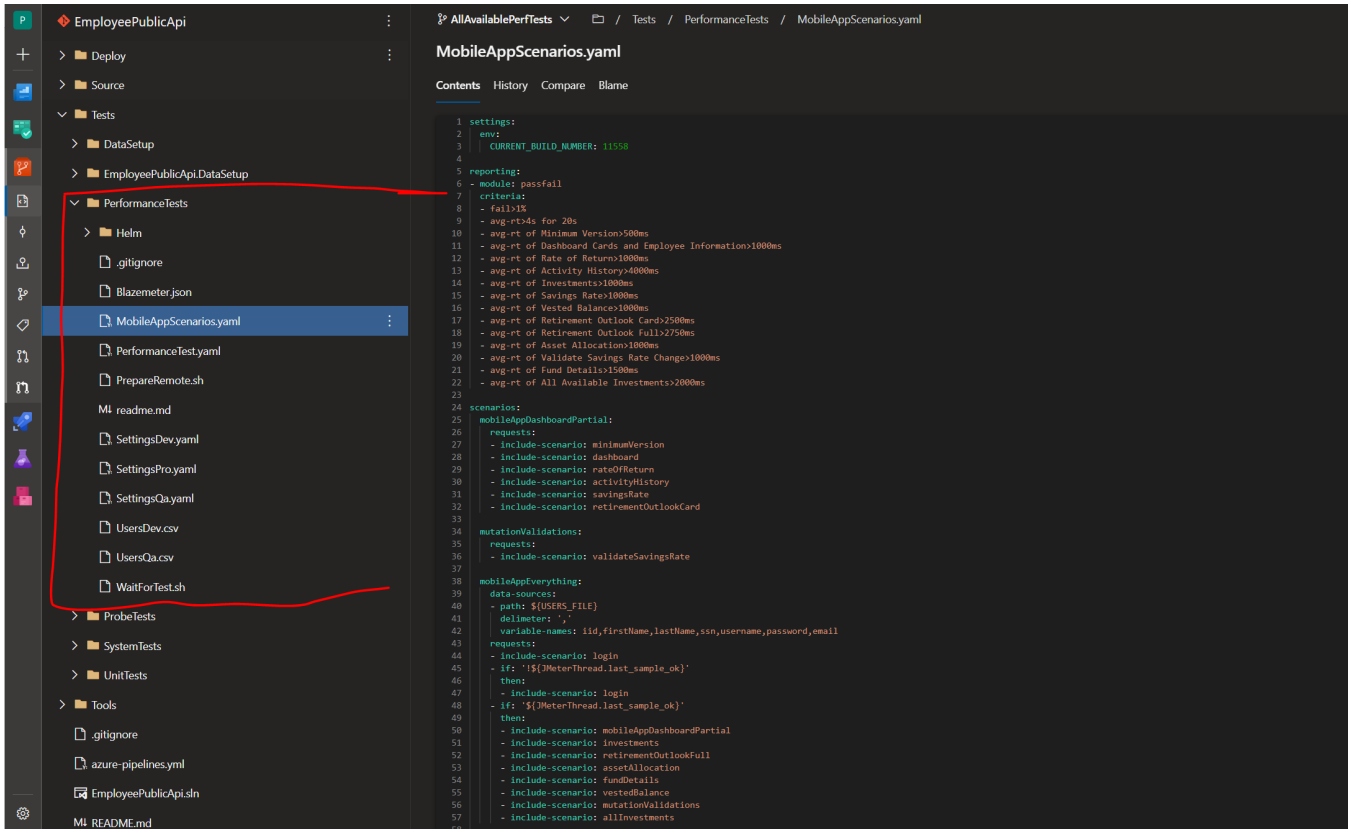
Test Organization

One of the best features of Taurus is the ability to separate all of your configuration files. Instead of one big Jmeter XML file that is virtually impossible to decipher in source control, you can split up your files in Taurus in such a way that is easily readable and maintainable.

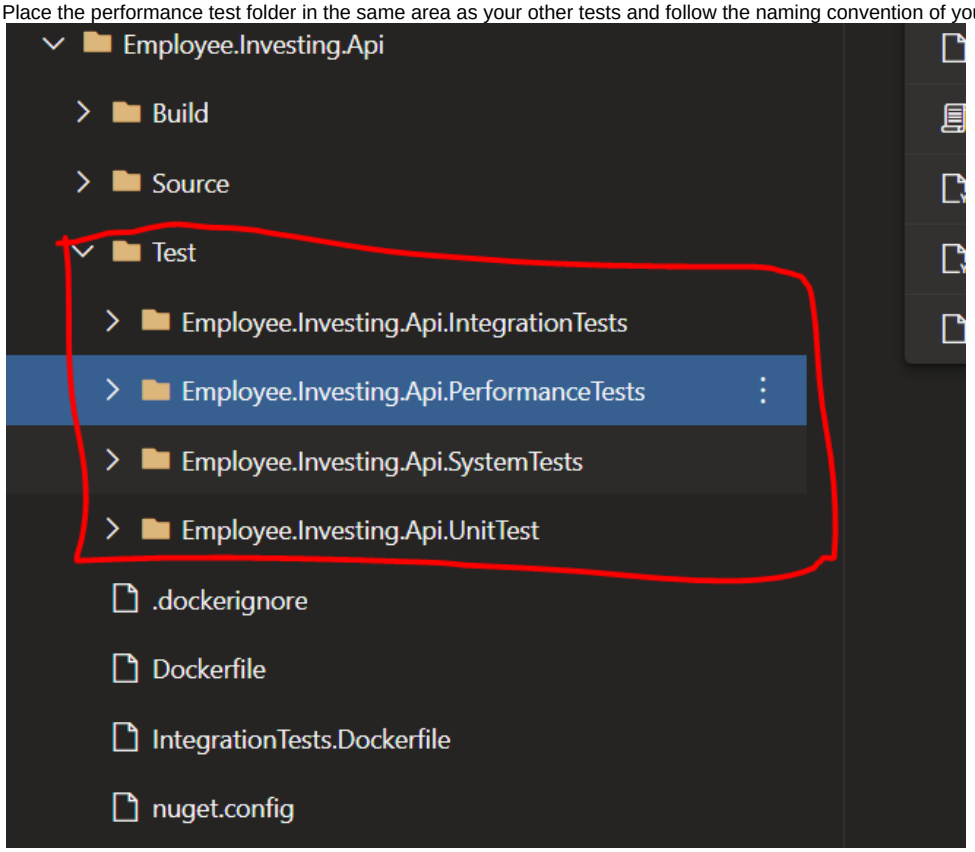
See this sample repo for the Employee Public API to see how the tests are organized.

https://tfs.ascensus.com/tfs/PSG/Portfolio/_git/EmployeePublicApi?path=%2FTests%2FPerformanceTests



There are no strict rules but here are some guidelines

- Place the performance test folder in the same area as your other tests and follow the naming convention of your git repo



- Put all of your environmental settings into different files. In the example above, you can see there is a SettingsDev.yaml, SettingsQa.Yaml, and SettingsPro.yaml. This allows you to easily switch environments by simply swapping whatever settings file you want to run in the bzt command

Running in QA

```
bzt PerformanceTest.yaml MobileAppScenarios.yaml SettingsQa.yaml
```

Running in Production

```
bzt PerformanceTest.yaml MobileAppScenarios.yaml SettingsPro.yaml
```

- Have one or more files dedicated to your scenarios only and no other settings. In the above example, all of the scenarios are in one file called MobileAppScenarios.yaml. The amount of scenarios is not overwhelming, thus it is easily maintained in one file.

- Have one file specifically for test execution settings only. In this case, the file is called PerformanceTest.yaml.

PerformanceTest.yaml

Contents History Compare Blame

```yaml
settings:
  artifacts-dir: TestArtifacts

execution:
- concurrency: 1
  ramp-up: 30s
  hold-for: 600s
  scenario: mobileAppEverything
```

- Utilize the include-scenario directive to promote reusability. The biggest downside with Jmeter is you cannot reuse anything. Everything has to be duplicated everywhere. The beautify of Taurus is that you can nest scenarios into other scenarios, which makes this clean and tidy.

```yaml
scenarios:
  mobileAppDashboardPartial:
    requests:
    - include-scenario: minimumVersion
    - include-scenario: dashboard
    - include-scenario: rateOfReturn
    - include-scenario: activityHistory
    - include-scenario: savingsRate
    - include-scenario: retirementOutlookCard

  mutationValidations:
    requests:
    - include-scenario: validateSavingsRate

  mobileAppEverything:
    data-sources:
    - path: ${USERS_FILE}
      delimeter: ','
      variable-names: iid,firstName,lastName,ssn,username,password,email
    requests:
    - include-scenario: login
    - if: '!${JMeterThread.last_sample_ok}'
      then:
      - include-scenario: login
    - if: '${JMeterThread.last_sample_ok}'
      then:
      - include-scenario: mobileAppDashboardPartial
      - include-scenario: investments
      - include-scenario: retirementOutlookFull
      - include-scenario: assetAllocation
      - include-scenario: fundDetails
      - include-scenario: vestedBalance
      - include-scenario: mutationValidations
      - include-scenario: allInvestments
```